🎭 Playwright CLI Commands — The Complete Cheat Sheet | Naveen Automation Labs

▶️ RUNNING TESTS

Purpose	Command Syntax	Example
Run all tests	`npx playwright test`	`npx playwright test`
Run a specific test file	`npx playwright test <filename>`	`npx playwright test tests/login.spec.ts`
Run multiple specific test files	`npx playwright test <file1> <file2>`	`npx playwright test login.spec.ts cart.spec.ts`
Run a test using test name pattern (-g)	`npx playwright test -g "name"`	`npx playwright test -g "should login successfully"`
Run specific test using line number	`npx playwright test <file>:<line>`	`npx playwright test tests/login.spec.ts:15`
Run tests in a specific directory	`npx playwright test <dir>/`	`npx playwright test tests/api/`
Run tests with a specific tag (@tag)	`npx playwright test --grep "@tag"`	`npx playwright test --grep "@smoke"`
Run tests except those with a tag	`npx playwright test --grep-invert "@tag"`	`npx playwright test --grep-invert "@wip"`
Combine grep (AND logic)	`npx playwright test --grep "(?=.*@smoke)(?=.*login)"`	`npx playwright test --grep "(?=.*@smoke)(?=.*login)"`
Run only the last failed tests	`npx playwright test --last-failed`	`npx playwright test --last-failed`

💡 **TIP:** *Use **-g** to filter by test name, **--grep** for tags. Combine both: **npx playwright test -g "login" --grep "@smoke"***

🌐 BROWSER, PROJECT & CONFIG

Purpose	Command Syntax	Example
Run using a specific project from config	`npx playwright test --project=<project>`	`npx playwright test --project=chromium`
Run on multiple projects	`npx playwright test --project=a --project=b`	`npx playwright test --project=chromium --project=firefox`
Run tests in headed mode (see browser)	`npx playwright test --headed`	`npx playwright test --headed`
Run using a specific config file	`npx playwright test --config=<file>`	`npx playwright test --config=playwright.dev.config.ts`
Use environment variables	`ENV_VAR=value npx playwright test`	`BASE_URL=https://dev.site.com npx playwright test`
Set base URL from CLI	`npx playwright test --config ... (use env)`	`BASE_URL=https://staging.app.com npx playwright test`

⚙️ PARALLEL EXECUTION, WORKERS & RETRIES

Purpose	Command Syntax	Example
Run tests in parallel (default)	`npx playwright test`	`npx playwright test`
Run tests sequentially (1 worker)	`npx playwright test --workers=1`	`npx playwright test --workers=1`
Set specific number of workers	`npx playwright test --workers=<n>`	`npx playwright test --workers=4`
Use 50% of available CPU cores	`npx playwright test --workers=50%`	`npx playwright test --workers=50%`
Retry failed tests	`npx playwright test --retries=<n>`	`npx playwright test --retries=2`

Stop after first failure	`npx playwright test --max-failures=1`	`npx playwright test --max-failures=1`
Stop after N failures (fail fast)	`npx playwright test -x`	`npx playwright test -x`
Set global timeout per test	`npx playwright test --timeout=<ms>`	`npx playwright test --timeout=60000`
Set global expect timeout	`npx playwright test --expect-timeout=<ms>`	`npx playwright test --expect-timeout=10000`

💡 **TIP:** *For CI/CD use **--retries=2 --workers=50%** for stability. Use **-x** during development to fail fast.*

🔀 SHARDING (Split Tests Across Machines)

Purpose	Command Syntax	Example
Run shard 1 of 4 (CI machine 1)	`npx playwright test --shard=<current>/<total>`	`npx playwright test --shard=1/4`
Run shard 2 of 4 (CI machine 2)	`npx playwright test --shard=2/4`	`npx playwright test --shard=2/4`
Merge blob reports from all shards	`npx playwright merge-reports --reporter=html <dir>`	`npx playwright merge-reports --reporter=html ./blob-report`

💡 **TIP:** *In GitHub Actions, use a matrix strategy with **shard: [1/4, 2/4, 3/4, 4/4]** to split tests across 4 runners for faster CI.*

📊 REPORTERS & REPORTS

Purpose	Command Syntax	Example
Use a specific reporter	`npx playwright test --reporter=<reporter>`	`npx playwright test --reporter=html`
Use multiple reporters	`npx playwright test --reporter=list,html`	`npx playwright test --reporter=dot,json`
Open the HTML report	`npx playwright show-report`	`npx playwright show-report`
Open report from custom directory	`npx playwright show-report <dir>`	`npx playwright show-report ./my-report`
Save JSON report to file	`PLAYWRIGHT_JSON_OUTPUT_NAME=<file> npx playwright test --reporter=json`	`PLAYWRIGHT_JSON_OUTPUT_NAME=results.json npx playwright test --reporter=json`

💡 **REPORTERS:** ***list*** *(default, verbose)* | ***dot*** *(minimal)* | ***line*** *(one line per test)* | ***html*** *(interactive)* | ***json*** *(machine-readable)* | ***junit*** *(CI/CD)* | ***blob*** *(sharding)*

🐛 DEBUGGING & UI MODE

Purpose	Command Syntax	Example
Run tests in debug mode (Playwright Inspector)	`npx playwright test --debug`	`npx playwright test --debug`
Debug a specific test file	`npx playwright test <file> --debug`	`npx playwright test tests/login.spec.ts --debug`
Run in UI mode (interactive test runner)	`npx playwright test --ui`	`npx playwright test --ui`
Open UI mode on a specific port	`npx playwright test --ui-port=<port>`	`npx playwright test --ui-port=8080`
Enable Playwright Inspector via env	`PWDEBUG=1 npx playwright test`	`PWDEBUG=1 npx playwright test`
Run with trace ON (for post-mortem debugging)	`npx playwright test --trace=on`	`npx playwright test --trace=on`
Trace only on first retry	`npx playwright test --trace=on-first-retry`	`npx playwright test --retries=2 --trace=on-first-retry`
Trace only on retries (retain on failure)	`npx playwright test --trace=retain-on-failure`	`npx playwright test --trace=retain-on-failure`

View a trace file	`npx playwright show-trace <trace.zip>`	`npx playwright show-trace test-results/trace.zip`

💡 **TIP:** *Use **--ui** for interactive development. Use **--trace=on-first-retry** in CI — captures traces only for flaky/failing tests (saves storage).*

🛠️ CODE GENERATION, INSTALL & UTILITIES

Purpose	Command Syntax	Example
Generate test code by recording actions	`npx playwright codegen <url>`	`npx playwright codegen https://demo.site.com`
Codegen with specific viewport size	`npx playwright codegen --viewport-size=<w>,<h> <url>`	`npx playwright codegen --viewport-size=375,812 https://demo.site.com`
Codegen emulating a device	`npx playwright codegen --device="<device>" <url>`	`npx playwright codegen --device="iPhone 14" https://demo.site.com`
Codegen with saved auth state	`npx playwright codegen --load-storage=<file> <url>`	`npx playwright codegen --load-storage=auth.json https://demo.site.com`
Save auth state during codegen	`npx playwright codegen --save-storage=<file> <url>`	`npx playwright codegen --save-storage=auth.json https://demo.site.com`
Install all browsers	`npx playwright install`	`npx playwright install`
Install a specific browser	`npx playwright install <browser>`	`npx playwright install chromium`
Install browsers + OS dependencies	`npx playwright install --with-deps`	`npx playwright install --with-deps`
Create a new Playwright project	`npm init playwright@latest`	`npm init playwright@latest`
Open a URL in a specific browser	`npx playwright open --browser=  <url>`	`npx playwright open --browser=firefox https://demo.site.com`
Take a screenshot of a page	`npx playwright screenshot <url> <output>`	`npx playwright screenshot https://demo.site.com shot.png`
Generate a PDF of a page	`npx playwright pdf <url> <output>`	`npx playwright pdf https://demo.site.com page.pdf`
Check Playwright version	`npx playwright --version`	`npx playwright --version`
List installed browsers	`npx playwright install --dry-run`	`npx playwright install --dry-run`

💡 **TIP:** *Use **codegen** to record tests quickly. Use **--save-storage** to capture auth state once, then **--load-storage** to skip login in future recordings.*

📸 SCREENSHOTS, VIDEO & ARTIFACTS

Purpose	Command Syntax	Example
Capture screenshot on failure	`npx playwright test --screenshot=only-on-failure`	`npx playwright test --screenshot=only-on-failure`
Capture screenshot for every test	`npx playwright test --screenshot=on`	`npx playwright test --screenshot=on`
Record video for every test	`npx playwright test --video=on`	`npx playwright test --video=on`
Record video only on first retry	`npx playwright test --video=on-first-retry`	`npx playwright test --video=on-first-retry`
Record video, retain only on failure	`npx playwright test --video=retain-on-failure`	`npx playwright test --video=retain-on-failure`
Set custom output directory for artifacts	`npx playwright test --output=<dir>`	`npx playwright test --output=./test-artifacts`

💡 **TIP:** *For CI/CD, use **--screenshot=only-on-failure --video=retain-on-failure --trace=on-first-retry** — captures everything you need to debug without wasting storage.*

🔄 SNAPSHOT & VISUAL TESTING

Purpose	Command Syntax	Example
Update snapshots (visual baselines)	`npx playwright test --update-snapshots`	`npx playwright test --update-snapshots`
Ignore snapshot expectation (pass even if different)	`npx playwright test --ignore-snapshots`	`npx playwright test --ignore-snapshots`

💡 **TIP:** *Run **--update-snapshots** after intentional UI changes to update the visual baselines. Always review diffs before committing.*

🚀 POWER COMBOS — Real-World CI/CD Commands

Scenario	Full Command
Smoke tests on Chrome (headed)	`npx playwright test --grep "@smoke" --project=chromium --headed`
CI pipeline (headless, retries, HTML report)	`npx playwright test --retries=2 --reporter=html --workers=50% --trace=on-first-retry`
Debug a single failing test	`npx playwright test tests/checkout.spec.ts:42 --debug --project=chromium`
Run regression excluding WIP	`npx playwright test --grep "@regression" --grep-invert "@wip" --retries=1`
Staging env, all browsers, fail fast	`BASE_URL=https://staging.app.com npx playwright test --project=chromium --project=firefox --project=webkit -x`
Full CI with shard, video & trace	`npx playwright test --shard=1/4 --retries=2 --video=retain-on-failure --trace=on-first-retry --reporter=blob`